Exercício 1: acompanhe o algoritmo

Considere o vetor: **[7, 3, 5, 2]**

Faça manualmente a primeira passagem de cada algoritmo:

- Bubble Sort;
- Selection Sort;
- Insertion Sort.

Para cada um, responda:

1. Quais comparações são realizadas?
2. Quais elementos mudam de posição?
3. Qual é o vetor após a primeira passagem ou iteração?
4. Qual algoritmo realiza uma seleção explícita do menor elemento?

Exercício 2: escolha a propriedade correta

Associe cada situação à propriedade mais importante.

1. Ordenar funcionários por salário, preservando a ordem anterior entre salários iguais.
   - ☐ Estabilidade
   - ☐ Uso de pilha
   - ☐ Pior caso quadrático

2. Ordenar um vetor sem precisar de um vetor auxiliar proporcional a n.
   - ☐ Estabilidade
   - ☐ In-place
   - ☐ Caso médio

3. Garantir tempo Θ(n log n) mesmo no pior caso.
   - ☐ Heap Sort ou Merge Sort
   - ☐ Selection Sort
   - ☐ Quick Sort clássico

Exercício 3: análise comparativa

Considere uma entrada de tamanho ***n***.
Um sistema precisa ordenar os dados frequentemente. Os requisitos são:

- o pior caso deve ser $\Theta(n \log n)$;
- a memória auxiliar deve ser pequena;
- a estabilidade não é obrigatória.

Perguntas:
a) Qual algoritmo estudado atende a esses requisitos?
b) Qual é a complexidade do melhor caso, do caso médio e do pior caso?
c) Por que o ***Quick Sort clássico*** não oferece a mesma garantia no pior caso?
d) Qual alternativa estável você consideraria se a preservação da ordem dos registros se tornasse obrigatória?

Exercício 4 — Bubble Sort: comparações e trocas

Considere o vetor:
v = [42, 17, 8, 33, 25, 11, 49, 6, 28, 14]

Execute manualmente o Bubble Sort para ordenar o vetor em ordem crescente.

a) Apresente o vetor após cada comparação realizada na primeira passagem.
b) Informe quantas comparações e trocas foram realizadas na primeira passagem.
c) Apresente o vetor ao final de cada passagem do algoritmo, até a ordenação completa.
d) Quantas comparações são realizadas no total pela implementação convencional sem interrupção antecipada?
e) Determine as complexidades de tempo do melhor caso, do caso médio e do pior caso dessa implementação.
f) Explique como uma condição de parada antecipada pode alterar a complexidade do melhor caso.

Exercício 5 — Insertion Sort: deslocamentos e custo

Considere o vetor:
v = [31, 12, 45, 7, 26, 18, 39, 4, 22, 15]

Execute manualmente o **Insertion Sort**.

a) Apresente o vetor após cada iteração do laço externo.
b) Identifique os elementos deslocados em cada iteração.
c) Quantos deslocamentos são realizados durante a inserção do valor 4?
d) Quantas comparações entre elementos são realizadas durante toda a execução? Considere separadamente as comparações que envolvem elementos do vetor e as verificações de limite.
e) Determine o melhor caso e o pior caso assintóticos do Insertion Sort.
f) Explique por que o Insertion Sort pode ser adequado para vetores pequenos ou quase ordenados.

Exercício 6 — Selection Sort: número de comparações e trocas

Considere o vetor:
v = [29, 13, 37, 5, 41, 19, 2, 34, 11, 23]

Execute o **Selection Sort** em ordem crescente.

a) Apresente o vetor após cada iteração do laço externo.
b) Indique o menor elemento encontrado em cada iteração e sua posição antes da troca.
c) Quantas comparações entre elementos são realizadas durante a execução completa?
d) Quantas trocas efetivas são realizadas? Não conte a atribuição de troca quando o menor elemento já estiver na posição correta.
e) Determine as complexidades de tempo do melhor caso, do caso médio e do pior caso.
f) Compare o número de trocas do **Selection Sort** com o comportamento do **Bubble Sort**.

Exercício 7 — Comparando algoritmos quadráticos

Considere os vetores:
v1 = [16, 9, 28, 3, 21, 7, 35, 12, 5, 24]
v2 = [3, 5, 7, 9, 12, 16, 21, 24, 28, 35]

Implemente **Bubble Sort, Selection Sort e Insertion Sort** sem utilizar sort() ou sorted().

a) Ordene os dois vetores com cada algoritmo.
b) Conte as comparações entre elementos e as trocas ou deslocamentos realizados.
c) Para o Bubble Sort, considere duas versões: uma sem interrupção antecipada e outra que termina quando uma passagem não realiza trocas.
d) Compare o comportamento dos três algoritmos nos dois vetores.
e) Explique por que os três algoritmos podem apresentar o mesmo pior caso assintótico, mas comportamentos diferentes para uma entrada já ordenada.
f) Identifique quais implementações são *in-place* e quais são estáveis.

Exercício 8 — Estabilidade na ordenação de registros

Uma empresa precisa ordenar seus funcionários por salário. Os registros estão representados por nome e salário:

```
funcionarios = [
    ("Marina", 4200),
    ("Rafael", 3500),
    ("Bianca", 4200),
    ("Carlos", 2800),
    ("Fernanda", 3500),
    ("Diego", 5100),
    ("Amanda", 2800),
    ("Lucas", 4200),
    ("Juliana", 3500),
    ("Pedro", 5100)
]
```

a) Ordene os registros por salário crescente, utilizando um algoritmo estável.
b) Apresente o resultado preservando a ordem original dos funcionários que possuem o mesmo salário.
c) Explique o conceito de estabilidade e sua importância nesse problema.
d) Indique quais dos seguintes algoritmos são estáveis nas implementações convencionais estudadas: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Heap Sort.
e) Explique por que a estabilidade pode ser importante quando os dados já foram ordenados anteriormente por outro critério.
f) É possível implementar um algoritmo originalmente instável de forma estável? Explique sua resposta.

Exercício 9 — Merge Sort: divisão e intercalação

Considere o vetor:
v = [38, 14, 27, 6, 42, 19, 3, 31, 11, 24]

Execute o **Merge Sort**, mostrando as etapas de divisão e intercalação.

a) Represente a árvore de recursão até chegar aos subvetores de um único elemento.
b) Apresente todos os subvetores obtidos no primeiro nível de divisão.
c) Mostre como os subvetores são ordenados e intercalados até formar o vetor final.
d) Quantas folhas existem na árvore de recursão?
e) Qual é a altura da árvore quando o tamanho da entrada é (n=10)? Considere a raiz como nível zero e a divisão até subvetores unitários.
f) Explique por que a intercalação de duas partes ordenadas exige, no máximo, trabalho linear no tamanho total das partes.

Exercício 10 — Merge Sort: recorrência e árvore de recursão

Considere o custo **do Merge Sort** tradicional para um vetor de tamanho (***n***):

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn$$

para (n>1), com (T(1)=d), em que (c) e (d) são constantes positivas.

a) Explique o significado dos dois termos recursivos e do termo (cn).
b) Para simplificar a análise, suponha inicialmente que (n) seja uma potência de 2 e escreva a recorrência na forma (**T(n)=2T(n/2)+cn**).
c) Expanda a recorrência por pelo menos quatro níveis.
d) Construa a árvore de recursão e determine o custo total de cada nível.
e) Determine a altura da árvore e o número de folhas.
f) Utilize os resultados para demonstrar que o tempo de execução é **Θ(nlog n).**
g) Explique por que a divisão de um vetor com tamanho ímpar não altera a ordem assintótica do Merge Sort.

Exercício 11 — Quick Sort: partição e escolha do pivô

Considere o vetor:
v = [44, 18, 32, 7, 26, 39, 12, 48, 21, 5]

Utilize a implementação clássica do ***Quick Sort*** com partição de Lomuto, na qual o último elemento da região é escolhido como pivô.

a) Execute a primeira partição e apresente o vetor resultante.
b) Identifique a posição final do pivô após a primeira partição.
c) Indique os subproblemas gerados pelas chamadas recursivas.
d) Continue a execução até ordenar o vetor.
e) Represente a árvore de chamadas recursivas produzida por essa execução.
f) Explique como a escolha do pivô pode produzir partições equilibradas ou desequilibradas.
g) Indique as complexidades do melhor caso, do caso médio e do pior caso do Quick Sort clássico.

Observação: apresente os vetores após as trocas realizadas pela implementação escolhida. Outras implementações de partição podem produzir sequências intermediárias diferentes.

Exercício 12 — Quick Sort: recorrências e profundidade recursiva

Considere duas situações teóricas do Quick Sort clássico.

Situação A — Partições equilibradas
O pivô divide o problema em duas partes de tamanhos aproximadamente iguais.

Situação B — Partições extremamente desequilibradas
Uma das partes contém (n-1) elementos e a outra é vazia.

a) Escreva a recorrência de tempo para cada situação, considerando custo linear de particionamento.
b) Resolva a recorrência da Situação A.
c) Expanda a recorrência da Situação B até chegar ao caso base.
d) Determine a complexidade de tempo de cada situação.
e) Compare a profundidade da recursão nos dois casos.
f) Explique por que o **Quick Sort** pode ter tempo médio **$\Theta(n \log n)$**, mas tempo de pior caso **$\Theta(n^2)$**.
g) Relacione essa análise aos métodos de iteração, árvore de recursão e substituição estudados na aula anterior.

Exercício 13 — Heap Sort: construção e ordenação

Considere o vetor:

v = [36, 15, 42, 8, 27, 4, 31, 19, 11, 23]

Utilize o Heap Sort para ordenar o vetor em ordem crescente.

a) Represente o vetor como uma árvore binária completa.
b) Construa um heap máximo e mostre as alterações necessárias para satisfazer a propriedade de heap.
c) Apresente o vetor após a construção do heap máximo.
d) Mostre cada troca entre a raiz e o último elemento da região ativa.
e) Após cada troca, apresente o resultado da restauração do heap.
f) Mostre o vetor após cada extração até a ordenação completa.
g) Explique por que a construção inicial do heap custa **Θ(n)**, mas o custo total do **Heap Sort** é **Θ(nlog n)**.
h) Indique se o **Heap Sort** convencional é *in-place* e estável.

Exercício 14 — Escolha do algoritmo: tempo, memória e estabilidade

Uma empresa possui três necessidades diferentes:

- **Cenário A:** ordenar um vetor de 10 elementos, normalmente quase ordenado.
- **Cenário B:** ordenar milhões de registros, preservando a ordem relativa dos registros com a mesma chave.
- **Cenário C:** ordenar um grande vetor em memória limitada, exigindo garantia de tempo **Θ(nlog n)**, no pior caso e espaço auxiliar constante na implementação convencional.

Para cada cenário:

a) Escolha o algoritmo mais adequado entre Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Heap Sort.
b) Justifique sua escolha considerando tempo, memória auxiliar e estabilidade.
c) Apresente uma alternativa e explique sua principal desvantagem.
d) Explique por que a complexidade assintótica não é o único critério para escolher um algoritmo.
e) No cenário B, explique por que a estabilidade pode ser mais importante do que minimizar o espaço auxiliar.

Exercício 15 — Desafio integrador: código, recorrência e complexidade

Considere o seguinte algoritmo de ordenação recursiva:

```
def ordenar_recursivo(v):
    if len(v) <= 1:
        return v
    meio = len(v) // 2
    esquerda = ordenar_recursivo(v[:meio])
    direita = ordenar_recursivo(v[meio:])

    return intercalar(esquerda, direita)
```

Considere que intercalar(esquerda, direita) recebe duas listas ordenadas e produz uma lista ordenada em tempo linear em relação ao total de elementos.

Utilize o vetor: **v = [47, 16, 33, 2, 29, 41, 9, 25, 6, 38]**

a) Identifique o caso base e explique por que ele é necessário.
b) Execute o algoritmo conceitualmente, mostrando as divisões recursivas e as intercalações.
c) Quantas chamadas recursivas são realizadas em cada chamada não terminal?
d) Escreva a recorrência de tempo do algoritmo.
e) Resolva a recorrência utilizando a árvore de recursão ou o Método Mestre, explicando as hipóteses necessárias.
f) Determine a complexidade de tempo no melhor caso, no caso médio e no pior caso.
g) Analise o uso de memória da implementação, considerando a pilha de recursão e as listas auxiliares criadas por fatiamento e intercalação.
h) Compare esse algoritmo com o Quick Sort clássico quanto a tempo no pior caso, estabilidade e uso de memória.
i) Explique como os conceitos de caso base, chamadas recursivas, recorrência e árvore de recursão ajudam a justificar matematicamente a complexidade do algoritmo.